Writing for the sidenote layout

Layout notes

September 2026

What this document is

This is a short article written in Markdown. It exists to show, on itself, how notes are written so that they land in the margin, level with the mark that calls them.[1] The same file builds to a web page and to a PDF, and in both the page is divided the same way: the left two thirds hold the text, the right third holds the notes.

You write in ordinary pandoc Markdown. Headings start with `#`, emphasis uses `*asterisks*`, and a paragraph is any run of lines separated by a blank line.[2] Nothing about the layout is written into the text. The layout lives in a few small files in folders beside this one, and the text only ever says *put a note here*.

[1] A note like this one. Its first line sits on the line of text that carries its number.

[2] Pandoc's own manual describes the full syntax. Only footnotes and headings matter for the layout.

Writing a single note

A note has two halves. In the sentence, at the point where the number should appear, you write a label in square brackets with a caret: `[^label]`. Anywhere else in the file, usually just after the paragraph, you write the same label followed by a colon and the text of the note.[3] The label is for you. It can be a word or a number and it is never printed; the notes are numbered in the order their marks appear.

```
Text with a note.[^cost] More text.

[^cost]: The text of the note.
```

A note may run to more than one paragraph. Indent every line after the first by four spaces and separate the paragraphs with a blank line, as the note on this sentence does.[4] Keep notes short where you can. A note that runs longer than the paragraph it hangs from will push the notes below it down the page, and in the PDF it may not fit before the page ends.

[3] The label here is `label`. In the source it reads `[^label]`.

[4] This note has two paragraphs.

The second paragraph is indented by four spaces in the source so that pandoc knows it still belongs to the note.

Notes and page breaks

The web page has no pages, so a note is never moved. The PDF does have pages, and a note whose mark falls near the foot of a page may not have room below the mark. When that happens the note is pulled up the margin just far enough to fit, so that its last line sits level with the last line of text on the page. It is not split. Its mark stays where it is in the text.[5] Only a note taller than the whole margin is carried to the next page. If you would rather the

[5] This note is written to fall across a page break in the PDF. It is long enough not to fit in the margin left below its mark, and LaTeX cannot split a margin note between pages, so the marginfix package pulls the whole note up until it fits. The number in the text does not move. If you change the paper size or the font, this note may land somewhere else, and the paragraph that carries it may need lengthening or shortening to straddle a break again.

note were level with its mark, shorten the note or move the sentence. There is no markup for forcing it.

This is the one place the two outputs differ. On the web the note on the previous sentence sits level with its mark. In the PDF, if its mark lands near the foot of a page, the note starts above the mark.

Consecutive notes

When several marks fall close together, their notes cannot all sit level with their marks without overlapping. The rule is that each note sits as high as it can without touching the note above it. So the first is level with its mark and the rest queue below it. This sentence carries three marks in a row, one on the first clause,[6] one on the second,[7] and one at the end,[8] and the margin shows the queue.

Nothing special is written to get this. Three marks, three note bodies, and the layout does the rest. The queue resolves itself: once the text has moved on far enough, the next note is level with its mark again.[9]

Fonts

Both outputs use the same ordered list of typefaces and take the first one that is installed on the machine doing the build. The list is written once in each of the two style files.[10] The order is:

1. Arbitare Sans, at weight 400
2. Serif
3. Sans Serif
4. Helvetica Neue

On the web the second and third entries are the browser's generic families, and a generic family always matches. So a browser that lacks Arbitare Sans will stop at *Serif* and never reach Helvetica Neue.[11] LaTeX has no generic families, so the PDF build stands in Latin Modern Roman for *Serif* and Latin Modern Sans for *Sans Serif*; both ship with every TeX installation and so, as on the web, the fourth entry is only reached if the TeX fonts are missing.

Citing sources

References live in a BibTeX file named in the metadata block, and a citation in the text is the entry's key in square brackets with an at sign: `[@brandi1963]`. Pandoc reads the BibTeX file itself, turns the key into a formatted citation and collects everything cited into a reference list at the end. No separate bibtex or biber run is involved, and the two outputs format their citations identically. The style file named in the metadata decides the shape of both. This document uses the Chicago notes style, so every citation becomes a note in the margin, like this one.[12] Page numbers and other locators follow the key inside the brackets.[13] A second citation of the same work is shortened automatically.[14]

A citation may also sit inside an ordinary note, where it is set as text rather than as a note of its own.[15] Several keys separated by semicolons make one note.[16]

[6] The first of three notes on one sentence. It is level with its mark because nothing sits above it.

[7] The second. Its mark is on the same line as the first, so it waits below the first note.

[8] The third, queued below the second for the same reason.

[9] Back in step. This note's mark is far enough down the page that the queue above has cleared.

[10] `css/style.css` for the web page, `tex/preamble.tex` for the PDF.

[11] If Helvetica Neue is meant as a real fallback, put it before the generic names in both files.

[12] Cesare Brandi, *Teoria del restauro* (Edizioni di Storia e Letteratura, 1963).

[13] Salvador Muñoz Viñas, *Contemporary Theory of Conservation* (Elsevier Butterworth-Heinemann, 2005), 91–93.

[14] Brandi, *Teoria del restauro*, 34.

[15] See Barbara Appelbaum, *Conservation Treatment Methodology* (Butterworth-Heinemann, 2007) on the same point. Written as `@appelbaum2007`, without brackets, the citation is set as a name in running text.

[16] Appelbaum, *Conservation Treatment Methodology*; Robert L. Feller, *Accelerated Aging: Photochemical and Thermal Aspects* (Getty Conservation Institute, 1994).

Delete the `csl` line from the metadata and pandoc falls back to its built-in Chicago author–date style, which puts citations in the text as parenthetical author and year and leaves the margin to your own notes. Either way the reference list is placed wherever the empty `refs` block below sits in the source; put it under a heading of your choice.

Give any entry whose title is not in English a `langid` field, as the Brandi entry has. Without it the style capitalises the title as if it were English, so *Teoria del restauro* would come out as *Teoria Del Restauro*.[17]

[17] `langid = {italian}` in the BibTeX entry. BibLaTeX uses the same field name, so a bibliography that already carries it needs nothing more.

```
Brandi set out the argument.[@brandi1963, 34]

# References

::: {#refs}
:::
```

Building

Run the driver script on the Markdown file. It runs pandoc twice, once for each output, and writes both into `output/`:

```
./build.py README.md            # output/README.html and output/README.pdf
./build.py README.md -t pdf     # one output only
./build.py README.md --papersize a4 --verbose
./build.py README.md -- --toc   # anything after -- goes straight to pandoc
./build.py --help
```

The script needs only Python and pandoc; its `uv` header lets `uv run` start it with no setup. Every option is a pandoc option of the same name with a default that points at the files in this folder:

```
build.py
css/style.css                      web page layout and fonts
lua/sidenotes.lua                  pandoc filter, footnotes to sidenotes
tex/preamble.tex                   PDF layout, fonts and margin notes
tex/chicago-notes-bibliography.csl citation style
example/readme.bib                 references for this document
output/                            built files
```

With the defaults filled in, the two pandoc commands it runs are:

```
pandoc README.md --citeproc --standalone --lua-filter=lua/sidenotes.lua \
       --css=css/style.css --embed-resources -o output/README.html
pandoc README.md --citeproc --pdf-engine=lualatex \
       --include-in-header=tex/preamble.tex -o output/README.pdf
```

The Lua filter rewrites each footnote as an inline span for the web page. The preamble sets the page geometry, the font fallback and the margin note packages for the PDF.[18] Neither touches the Markdown.

[18] The PDF build records which font it settled on. Run the script with `--verbose` and look for a line beginning `[fontfallback]`.

References

Appelbaum, Barbara. *Conservation Treatment Methodology.* Butterworth-Heinemann, 2007.

Brandi, Cesare. *Teoria del restauro.* Edizioni di Storia e Letteratura, 1963.

Feller, Robert L. *Accelerated Aging: Photochemical and Thermal Aspects*. Getty Conservation Institute, 1994.

Muñoz Viñas, Salvador. *Contemporary Theory of Conservation*. Elsevier Butterworth-Heinemann, 2005.